BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

> **Objective & Lecture Mapping:** In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

> To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an* `__init__` *method to store history.*

Example Logic: `IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward`

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent` .
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)` , first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
   Example Logic: `IF wall_ahead AND left_is_visited THEN turn_right`
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

> Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A Table-Driven Agent requires a pre-programmed lookup table for every possible percept sequence. In complex environments like Chess, this causes a combinatorial explosion. Chess has roughly 10^{43} possible board configurations, and the number of sequential moves is even higher. As the agent's lifetime increases, the percept history grows exponentially, making the table mathematically impossible to compute, store in memory, or search efficiently. We write Agent Programs instead to

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

The "Condition-Action Rules" are mapped directly to the if-elif-else block in the sense_and_act method:

if percept.get('wall_ahead'): represents the Condition (evaluating the immediate sensory input).

return 'Left' represents the Action (the hardcoded reflex triggered by that condition).
This perfectly mirrors the lecture's definition of a Simple Reflex Agent: it maps current percepts to actions via direct rules, bypassing any deeper reasoning or historical context.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The agent failed because it evaluates the world purely based on its current sensory snapshot. Because the environment is partially observable (it can only see the cell directly ahead, not the global map), a corner looks exactly the same as a flat wall. Because it lacks percept history (no internal memory), it doesn't remember that it was standing in this exact spot 2 steps ago. When presented with the identical local percept, it fires the exact same condition-action rule over and over again, trapping it in

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent`. Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

In the ModelBasedAgent, these models are represented by the internal state tracking:

Sensor Model: By combining the immediate percept (wall_ahead) with the memory variable (self.consecutive_walls_hit), the agent deduces unobservable information. The raw sensor just says "wall," but the model deduces "I am stuck in a corner."

Transition Model: By tracking the last_action and updating state variables before choosing the next move, the agent understands that its previous actions (e.g., turning Left and moving Up) resulted in its current predicament, allowing it to transition to a new behavior (turning Right).

Finalize and Lock Lab 02 Documentation

FACULTY OF COMPUTING